

校園跑腿 APP - 完整程式源代碼

目錄

1. [核心類型定義](#)
 2. [全域狀態管理](#)
 3. [本地儲存工具](#)
 4. [首頁 - 探索任務](#)
 5. [發單頁面](#)
 6. [任務詳情頁面](#)
 7. [訂單管理頁面](#)
 8. [個人頁面](#)
 9. [評價頁面](#)
-

核心類型定義

文件： `lib/types.ts`

```
// 任務類型定義
export type TaskCategory = 'food' | 'document' | 'shopping' | 'other';
export type TaskStatus = 'open' | 'accepted' | 'completed' | 'cancelled' |
'expired';

export interface Task {
  id: string;
  title: string;
  description?: string;
  category: TaskCategory;
  pickupLocation: string;
  deliveryLocation: string;
  reward: number;
  deadline: string;
  status: TaskStatus;
  publisherId: string;
  publisherName: string;
  publisherRating: number;
  acceptorId?: string;
  acceptorName?: string;
  createdAt: string;
  updatedAt: string;
}

export interface Order {
  id: string;
  taskId: string;
  task: Task;
  publisherId: string;
  acceptorId: string;
  status: 'accepted' | 'completed' | 'cancelled';
  createdAt: string;
  updatedAt: string;
}

export interface Review {
  id: string;
  orderId: string;
  reviewerId: string;
  revieweeId: string;
  rating: number;
  comment: string;
  createdAt: string;
}
```

```
export interface UserProfile {
  id: string;
  name: string;
  rating: number;
  totalPublished: number;
  totalAccepted: number;
  completionRate: number;
  reviews: Review[];
}

// 任務分類定義
export const TASK_CATEGORIES = [
  { key: 'food' as const, label: '餐飲', icon: '🍔' },
  { key: 'document' as const, label: '文件', icon: '📄' },
  { key: 'shopping' as const, label: '購物', icon: '🛒' },
  { key: 'other' as const, label: '其他', icon: '📦' },
];

export const TASK_STATUS_LABELS: Record<TaskStatus, string> = {
  open: '待接單',
  accepted: '進行中',
  completed: '已完成',
  cancelled: '已取消',
  expired: '已過期',
};

export const TASK_STATUS_COLORS: Record<TaskStatus, string> = {
  open: '#10B981',
  accepted: '#3B82F6',
  completed: '#8B5CF6',
  cancelled: '#EF4444',
  expired: '#6B7280',
};
```

全域狀態管理

文件：lib/app-context.tsx

```

import React, { createContext, useContext, useEffect, useReducer,
useCallback } from 'react';
import { Task, Order, Review, UserProfile } from './types';
import {
  getTasks, saveTasks, addTask, updateTask,
  getOrders, addOrder, updateOrder,
  getReviews, addReview,
  getUser, saveUser,
  generateId,
  removeExpiredTasks,
} from './storage';

interface AppState {
  tasks: Task[];
  orders: Order[];
  reviews: Review[];
  user: UserProfile;
  loading: boolean;
}

const DEFAULT_USER: UserProfile = {
  id: 'local_user',
  name: '我',
  rating: 5.0,
  totalPublished: 0,
  totalAccepted: 0,
  completionRate: 100,
  reviews: [],
};

type AppAction =
  | { type: 'SET_LOADING'; payload: boolean }
  | { type: 'SET_TASKS'; payload: Task[] }
  | { type: 'ADD_TASK'; payload: Task }
  | { type: 'UPDATE_TASK'; payload: Task }
  | { type: 'DELETE_TASK'; payload: string }
  | { type: 'SET_ORDERS'; payload: Order[] }
  | { type: 'ADD_ORDER'; payload: Order }
  | { type: 'UPDATE_ORDER'; payload: Order }
  | { type: 'SET_REVIEWS'; payload: Review[] }
  | { type: 'ADD_REVIEW'; payload: Review }
  | { type: 'SET_USER'; payload: UserProfile }
  | { type: 'UPDATE_USER_NAME'; payload: string };

function reducer(state: AppState, action: AppAction): AppState {

```

```

switch (action.type) {
  case 'SET_LOADING':
    return { ...state, loading: action.payload };
  case 'SET_TASKS':
    return { ...state, tasks: action.payload };
  case 'ADD_TASK':
    return { ...state, tasks: [action.payload, ...state.tasks] };
  case 'UPDATE_TASK':
    return {
      ...state,
      tasks: state.tasks.map((t) => (t.id === action.payload.id ?
action.payload : t)),
    };
  case 'DELETE_TASK':
    return {
      ...state,
      tasks: state.tasks.filter((t) => t.id !== action.payload),
    };
  case 'SET_ORDERS':
    return { ...state, orders: action.payload };
  case 'ADD_ORDER':
    return { ...state, orders: [action.payload, ...state.orders] };
  case 'UPDATE_ORDER':
    return {
      ...state,
      orders: state.orders.map((o) => (o.id === action.payload.id ?
action.payload : o)),
    };
  case 'SET_REVIEWS':
    return { ...state, reviews: action.payload };
  case 'ADD_REVIEW':
    return { ...state, reviews: [action.payload, ...state.reviews] };
  case 'SET_USER':
    return { ...state, user: action.payload };
  case 'UPDATE_USER_NAME':
    return { ...state, user: { ...state.user, name: action.payload } };
  default:
    return state;
}
}

interface AppContextValue {
  state: AppState;
  publishTask: (data: Omit<Task, 'id' | 'status' | 'publisherId' |
'publisherName' | 'publisherRating' | 'createdAt' | 'updatedAt'>) =>
Promise<void>;
}

```

```

    acceptTask: (taskId: string) => Promise<void>;
    deleteTask: (taskId: string) => Promise<void>;
    completeOrder: (orderId: string) => Promise<void>;
    cancelOrder: (orderId: string) => Promise<void>;
    submitReview: (orderId: string, revieweeId: string, rating: number,
comment: string) => Promise<void>;
    updateUserName: (name: string) => Promise<void>;
  }

const AppContext = createContext<AppContextValue | null>(null);

export function AppProvider({ children }: { children: React.ReactNode }) {
  const [state, dispatch] = useReducer(reducer, {
    tasks: [],
    orders: [],
    reviews: [],
    user: DEFAULT_USER,
    loading: true,
  });

  // 初始化資料並清除過期任務
  useEffect(() => {
    async function load() {
      await removeExpiredTasks();
      const [tasks, orders, reviews, user] = await Promise.all([
        getTasks(),
        getOrders(),
        getReviews(),
        getUser(),
      ]);
      dispatch({ type: 'SET_TASKS', payload: tasks });
      dispatch({ type: 'SET_ORDERS', payload: orders });
      dispatch({ type: 'SET_REVIEWS', payload: reviews });
      dispatch({ type: 'SET_USER', payload: user ?? DEFAULT_USER });
      dispatch({ type: 'SET_LOADING', payload: false });
    }
    load();

    // 每 5 分鐘清除過期任務
    const interval = setInterval(() => {
      removeExpiredTasks();
    }, 5 * 60 * 1000);

    return () => clearInterval(interval);
  }, []);

```

```

    const publishTask = useCallback(async (data: Omit<Task, 'id' | 'status' |
'publisherId' | 'publisherName' | 'publisherRating' | 'createdAt' |
'updatedAt'>) => {
      const now = new Date().toISOString();
      const task: Task = {
        ...data,
        id: generateId(),
        status: 'open',
        publisherId: state.user.id,
        publisherName: state.user.name,
        publisherRating: state.user.rating,
        createdAt: now,
        updatedAt: now,
      };
      await addTask(task);
      dispatch({ type: 'ADD_TASK', payload: task });
      // 更新用戶統計
      const updatedUser = {
        ...state.user,
        totalPublished: state.user.totalPublished + 1,
      };
      await saveUser(updatedUser);
      dispatch({ type: 'SET_USER', payload: updatedUser });
    }, [state.user]);

const acceptTask = useCallback(async (taskId: string) => {
  const task = state.tasks.find((t) => t.id === taskId);
  if (!task || task.status !== 'open') return;
  const now = new Date().toISOString();
  const updatedTask: Task = {
    ...task,
    status: 'accepted',
    acceptorId: state.user.id,
    acceptorName: state.user.name,
    updatedAt: now,
  };
  await updateTask(updatedTask);
  dispatch({ type: 'UPDATE_TASK', payload: updatedTask });
  // 建立訂單
  const order: Order = {
    id: generateId(),
    taskId,
    task: updatedTask,
    publisherId: task.publisherId,
    acceptorId: state.user.id,
    status: 'accepted',
  };

```

```

        createdAt: now,
        updatedAt: now,
    };
    await addOrder(order);
    dispatch({ type: 'ADD_ORDER', payload: order });
    // 更新用戶統計
    const updatedUser = {
        ...state.user,
        totalAccepted: state.user.totalAccepted + 1,
    };
    await saveUser(updatedUser);
    dispatch({ type: 'SET_USER', payload: updatedUser });
}, [state.tasks, state.user]);

const deleteTask = useCallback(async (taskId: string) => {
    const task = state.tasks.find((t) => t.id === taskId);
    if (!task || task.publisherId !== state.user.id) return; // 只允許發布者刪除
    await saveTasks(state.tasks.filter((t) => t.id !== taskId));
    dispatch({ type: 'DELETE_TASK', payload: taskId });
}, [state.tasks, state.user.id]);

const completeOrder = useCallback(async (orderId: string) => {
    const order = state.orders.find((o) => o.id === orderId);
    if (!order) return;
    const now = new Date().toISOString();
    const updatedOrder: Order = { ...order, status: 'completed', updatedAt:
now };
    const updatedTask: Task = { ...order.task, status: 'completed',
updatedAt: now };
    await updateOrder(updatedOrder);
    await updateTask(updatedTask);
    dispatch({ type: 'UPDATE_ORDER', payload: updatedOrder });
    dispatch({ type: 'UPDATE_TASK', payload: updatedTask });
}, [state.orders]);

const cancelOrder = useCallback(async (orderId: string) => {
    const order = state.orders.find((o) => o.id === orderId);
    if (!order) return;
    const now = new Date().toISOString();
    const updatedOrder: Order = { ...order, status: 'cancelled', updatedAt:
now };
    const updatedTask: Task = { ...order.task, status: 'cancelled',
updatedAt: now };
    await updateOrder(updatedOrder);
    await updateTask(updatedTask);

```



```

    dispatch({ type: 'UPDATE_ORDER', payload: updatedOrder });
    dispatch({ type: 'UPDATE_TASK', payload: updatedTask });
  }, [state.orders]);

  const submitReview = useCallback(async (orderId: string, revieweeId:
string, rating: number, comment: string) => {
    const review: Review = {
      id: generateId(),
      orderId,
      reviewerId: state.user.id,
      revieweeId,
      rating,
      comment,
      createdAt: new Date().toISOString(),
    };
    await addReview(review);
    dispatch({ type: 'ADD_REVIEW', payload: review });
  }, [state.user.id]);

  const updateUserName = useCallback(async (name: string) => {
    const updatedUser = { ...state.user, name };
    await saveUser(updatedUser);
    dispatch({ type: 'UPDATE_USER_NAME', payload: name });
  }, [state.user]);

  return (
    <AppContext.Provider value={{ state, publishTask, acceptTask,
deleteTask, completeOrder, cancelOrder, submitReview, updateUserName }}>
      {children}
    </AppContext.Provider>
  );
}

export function useApp() {
  const ctx = useContext(AppContext);
  if (!ctx) throw new Error('useApp must be used within AppProvider');
  return ctx;
}

```

本地儲存工具

文件：lib/storage.ts

本地儲存工具提供以下功能：

- 任務 CRUD 操作
- 訂單管理
- 評價管理
- 用戶資料管理
- 過期任務自動清除
- 時間格式化

主要函數：

- `getTasks()` - 獲取所有任務
 - `addTask(task)` - 新增任務
 - `updateTask(task)` - 更新任務
 - `saveTasks(tasks)` - 保存所有任務
 - `removeExpiredTasks()` - 清除過期任務
 - `isTaskExpired(deadline)` - 檢查任務是否過期
 - `formatDeadline(deadline)` - 格式化截止時間
-

首頁 - 探索任務

文件：`app/(tabs)/index.tsx`

功能：

- 顯示所有開放任務
- 按類別篩選（全部、餐飲、文件、購物）
- 搜尋任務
- 點擊任務進入詳情頁面
- 整合雲端 API 查詢任務

主要特性：

- 使用 tRPC 查詢雲端任務
 - 合併本地和雲端任務
 - 即時搜尋和篩選
 - 空狀態提示
-

發單頁面

文件：`app/(tabs)/publish.tsx`

功能：

- 填寫任務標題、描述
- 選擇任務類別
- 輸入取件和送達地點
- 設定報酬金額
- 設定截止時間（小時）
- 發布到本地和雲端

表單驗證：

- 標題必填
 - 地點必填
 - 報酬必須是有效數字
 - 截止時間必須大於 0
-

任務詳情頁面

文件：`app/task/[id].tsx`

功能：

- 顯示完整任務資訊
- 顯示發布者資訊和評分

- 非發布者可接受任務
- 發布者可刪除任務
- 顯示任務狀態

UI 元素：

- 返回按鈕
 - 狀態徽章
 - 任務類別
 - 報酬卡片
 - 地點資訊
 - 截止時間
 - 發布者卡片
 - 接受/刪除按鈕
-

訂單管理頁面

文件：`app/(tabs)/orders.tsx`

功能：

- 分頁顯示「我發出的」和「我接的」訂單
- 顯示訂單狀態
- 確認完成訂單
- 取消訂單
- 進入訂單詳情

訂單狀態：

- accepted（進行中）
 - completed（已完成）
 - cancelled（已取消）
-

個人頁面

文件：`app/(tabs)/profile.tsx`

功能：

- 顯示用戶名稱和評分
- 顯示發單和接單統計
- 完成率展示
- 修改暱稱
- 清除所有資料（測試用）

用戶統計：

- 總發單數
 - 總接單數
 - 完成率
 - 平均評分
-

評價頁面

文件：`app/review/[orderId].tsx`

功能：

- 提交星級評分（1-5 星）
- 填寫評價文字
- 評價提交確認

評價資訊：

- 訂單 ID
- 被評價人 ID
- 星級評分
- 評價文字

- 建立時間
-

技術棧

- 框架：React Native + Expo 54
 - 語言：TypeScript
 - 樣式：NativeWind (Tailwind CSS)
 - 路由：Expo Router
 - 狀態管理：React Context + useReducer
 - 本地儲存：AsyncStorage
 - API 整合：tRPC
 - 動畫：React Native Reanimated
 - 觸覺反饋：expo-haptics
-

專案結構

```
campus-errand/
├─ app/
│   ├─ (tabs)/
│   │   ├─ _layout.tsx          # 底部導覽配置
│   │   ├─ index.tsx           # 首頁 (探索任務)
│   │   ├─ publish.tsx         # 發單頁面
│   │   ├─ orders.tsx          # 訂單管理
│   │   └─ profile.tsx         # 個人頁面
│   ├─ task/
│   │   └─ [id].tsx            # 任務詳情
│   ├─ order/
│   │   └─ [id].tsx            # 訂單詳情
│   ├─ review/
│   │   └─ [orderId].tsx       # 評價頁面
│   └─ _layout.tsx            # 根佈局
├─ lib/
│   ├─ app-context.tsx         # 全域狀態管理
│   ├─ types.ts                # 類型定義
│   ├─ storage.ts              # 本地儲存工具
│   ├─ trpc.ts                 # tRPC 客戶端
│   └─ utils.ts                # 工具函數
├─ components/
│   ├─ screen-container.tsx    # 安全區域容器
│   ├─ ui/
│   │   └─ icon-symbol.tsx      # 圖示映射
│   └─ ...
├─ hooks/
│   ├─ use-colors.ts           # 主題色彩 hook
│   ├─ use-auth.ts            # 認證 hook
│   └─ ...
└─ assets/
    └─ images/
        ├─ icon.png            # APP 圖示
        ├─ splash-icon.png      # 啟動畫面圖示
        └─ ...
```

核心流程

發單流程

1. 用戶進入「發單」頁面
2. 填寫任務資訊（標題、類別、地點、報酬、截止時間）
3. 點擊「發布任務」
4. 任務同時保存到本地和雲端
5. 返回首頁，任務出現在列表中

接單流程

1. 用戶在首頁瀏覽任務
2. 點擊任務進入詳情頁面
3. 點擊「接受任務」
4. 系統建立訂單
5. 任務狀態變為「進行中」
6. 訂單出現在「我的訂單」頁面

完成流程

1. 用戶進入「我的訂單」
2. 點擊「我接的」分頁
3. 選擇要完成的訂單
4. 點擊「確認完成」
5. 訂單狀態變為「已完成」
6. 可以評價發布者

評價流程

1. 訂單完成後
2. 進入評價頁面

3. 選擇星級評分
 4. 填寫評價文字
 5. 提交評價
 6. 評價保存到系統
-

最佳實踐

狀態管理

- 使用 Context + useReducer 管理全域狀態
- 避免 prop drilling
- 使用 useCallback 優化效能

效能優化

- 使用 useMemo 避免不必要的重新計算
- 使用 FlatList 而不是 ScrollView + map
- 圖片使用 expo-image 自動快取

使用者體驗

- 提供觸覺反饋 (haptics)
- 顯示 loading 狀態
- 錯誤提示和確認對話框
- 空狀態提示

安全性

- 驗證所有輸入
 - 只允許發布者刪除自己的任務
 - 只允許相關人員操作訂單
-

後續改進方向

1. **雲端同步** - 整合 Supabase 實現多人協作
 2. **即時通知** - WebSocket 或 Firebase 即時更新
 3. **聯絡方式** - 新增 LINE ID 或電話欄位
 4. **任務排序** - 按報酬、時間排序
 5. **用戶認證** - Google OAuth 登入
 6. **支付系統** - Line Pay 或其他支付方式
 7. **地圖整合** - 顯示任務位置
 8. **推播通知** - 任務更新時發送通知
-

生成時間：2026-03-30 版本：1.0.0